



GumBall6900

How GUM BALL 6900 turns community conviction into an onchain portfolio

A plain-language walkthrough of the whole protocol: what GBX is, what signaling does, where revenue comes from, how acquisitions and redemptions work, and what the design explicitly does not guarantee.

VERSION

2.1.0

SOURCE COMMIT

uncommitted-...

DEPLOYMENT

Not deployed on any network. No signed deployment manifest exists.

DATE

2026-08-26

PROTOCOL STATUS

Development candidate implementing ADRs through ADR 0051; not approved for user funds.

INDEPENDENT AUDIT

V12 export received for the pinned commit; incomplete assurance package, three behaviors confirmed, no release approval.

Before you read on: this protocol is not deployed or approved for user funds. A V12 finding export targets `3ae171b`, but it is incomplete and not release-authorizing; independent disposition confirmed three behaviors requiring treatment. The current ADR 0051 batching and periphery changes are later and not covered by V12. This is not a live product, and nothing here is investment advice.

1. The central idea

Imagine an investment club with three unusual rules.

First, it has no manager. Members commit a token to express conviction — “we should be buying wrapped bitcoin” — and the club’s income flows toward whichever holdings carry the most committed conviction. Move your commitment, and the flow follows.

Second, it never negotiates a purchase. When it has money to spend, it holds a public auction: it offers its cash to anyone willing to hand over the target asset and lowers its asking price steadily until someone accepts. No appraiser, no price feed, no discretion.

Third, leaving is arithmetic rather than a request. Burn your membership tokens and the vault hands you your proportional share of whichever holdings you name. No redemption window, no gate, no manager who can say no.

That is GumBall6900. These are not policies people follow — they are smart contracts that execute. The manager is not honest; the manager does not exist.

2. Why ordinary funds require trust

Buying into a conventional fund means trusting a chain of promises: that the fund holds what the statement says, that the custodian really has the assets, that the fee calculation is right, that you can withdraw on the stated terms. Each is enforced by law and reputation — slowly, and after the fact.

Early onchain funds moved the assets on-chain but often kept the trust. Look closely at many “decentralized index” products and you find an admin key that can change holdings, an upgradeable contract that can change rules, a pause switch that can stop withdrawals, and an oracle that decides what everything is worth. Same promises, different paperwork.

GumBall6900’s premise is that if you remove *every* discretionary lever, what remains is either verifiable or absent. In this development tree the protocol has no upgrade path, proxy, pause switch, rescue or sweep function, arbitrary-call executor, migration route, price oracle, NAV calculation, rebalancing engine, or keeper role. The treasury has no owner at all.

The honest flip side: removing every lever also removes every repair. A bug cannot be patched. A deployment mistake cannot be corrected.

3. What GBX represents

GBX is the protocol’s token. It is an ordinary transferable ERC-20 — you can hold it, send it, and trade it.

GBX starts at zero supply. Every unit is created through **mining** — continuous issuance to whoever occupies the mine’s slots. There is no team, presale, treasury, or liquidity premint.

The crucial structural fact is that mint authority is handed to one contract, exactly once, at deployment, and then permanently locked. There is no way to add a second minter, replace the first, or reopen the handover. Burning GBX doesn't reopen it either. Supply reconciles exactly: total supply always equals everything ever minted minus everything ever burned.

There is **no supply cap**. The prospective issuance rate halves at fixed intervals measured from Mine deployment, but it settles at a permanent floor that is strictly positive rather than falling to zero. GBX inflates forever, slowly.

Holding GBX gives you two rights, and it is worth being precise about which is which:

- **Signal with it**, and you direct protocol revenue and earn rewards.
- **Burn it**, and you withdraw your share of the treasury.

Holding GBX in your wallet does neither of those things by itself.

4. What it means to signal

The protocol has **Strategies** — each one a standing mandate to acquire one particular asset. A Strategy for wrapped bitcoin. A Strategy for a staked-ETH token. And so on.

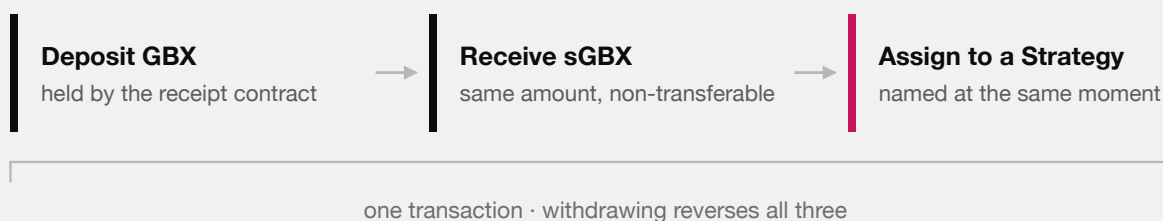
Signaling is a single atomic step with three parts that always happen together:

1. You deposit GBX into **SignalGBX**, ticker **sGBX**.
2. You receive exactly the same amount of sGBX.
3. That sGBX is committed to one Strategy you name.

Withdrawing reverses all three at once. There is deliberately **no in-between state**: you cannot hold sGBX that isn't committed to something. Every sGBX unit in existence is assigned to exactly one Strategy at all times.

SIGNALING IS ONE STEP, NOT THREE

All of this happens in a single transaction, or none of it happens.



There is deliberately no state in between. You cannot hold sGBX that is not pointed at something, so every unit of signal that exists is a unit of signal doing work.

DEPOSIT, RECEIPT, AND COMMITMENT ARE ONE ATOMIC STEP. THE ABSENCE OF AN IN-BETWEEN STATE IS WHAT MAKES THE AMOUNT OF SIGNAL THAT EXISTS AND THE AMOUNT DOING WORK THE SAME QUANTITY.

That is a real design choice with consequences, so it's worth stating plainly. It means every unit of signal that exists is a unit of signal doing work — there is no parked, idle, or purely symbolic position. The amount of sGBX in existence and the amount actually directing revenue are the same number, always.

sGBX is **non-transferable**. You cannot send it, sell it, or lend it. Any transfer attempt reverts.

You can spread your position across Strategies by signaling each one separately — 700 GBX to the bitcoin Strategy, 300 to the staked-ETH Strategy. These are absolute amounts, not percentages.

Three properties matter:

- **There is no lock-up, cooldown, or epoch.** You can add or remove signal in consecutive blocks or in one smart-wallet transaction.
- **You can always remove your signal.** Removal is bounded only by what you actually have committed to the selected Strategy.
- **Every signal change first settles the revenue that accrued under the old weights.** Reallocating never claws back or redirects money that already accrued to someone else. It only affects flow from that moment on.

Reallocation is a direct source removal followed by a destination addition. A smart account may compose both direct SignalGBX calls atomically, so an invalid destination rolls the complete wallet batch back; Resonance does not expose a separate move-only hook.

The complete user surface is four functions: scalar `addSignal` and `removeSignal`, plus optional struct-array `addSignalMany` and `removeSignalMany`. SignalGBX consumes no permit signature.

5. What sGBX is, and what it is not

sGBX is a **receipt**, not a second tradeable token. It exists to record two things: that you deposited GBX, and which Strategy you pointed it at. It cannot leave your wallet, and it has no market.

So there are two positions a person can hold, and they are not interchangeable:

POSITION	DIRECTS REVENUE?	EARNs STRATEGY REWARDS?	CAN REDEEM FROM THE FUND?
Holding liquid GBX in a wallet	No	No	Yes (by burning)
Signaling a Strategy with sGBX	Yes	Yes	No (withdraw first)

There is one more thing sGBX quietly does. It keeps **vote checkpoints** — a standing record of how much sGBX each account held at each past block, in the standard format governance systems read. Your first signal switches that recording on automatically, with no second transaction.

Nothing in the protocol reads those checkpoints today. They are there because the protocol expects an external governance system to be attached later, and that system will need them. Which system, and on what rules, is not decided — §13 is honest about what that means.

One property worth understanding now, because it does not go away: **checkpoints survive withdrawal**. Once a block has passed, the record of what you held at that block is permanent history, even if you withdrew everything afterwards. If a governance system is later attached that grants power based on past blocks, someone could borrow GBX, signal it, let a block pass, withdraw, and still carry the recorded weight. Whether that is exploitable depends entirely on the system chosen. It is tracked internally as finding G-01 and is one of the reasons that choice has not been rushed.

6. Where the money comes from

Protocol revenue arrives in **USDG**, a stablecoin the protocol neither issues nor controls. In the intended deployment it has six decimal places, which matters later for the arithmetic.

There is one protocol-defined revenue source.

Mining. The mine has exactly sixteen permanent slots. Whoever occupies a slot accrues GBX continuously at a fixed rate, minted when that slot's current tenure is replaced. To take a slot, you win its auction: the replacement price starts at some level and falls in a straight line to zero over one hour, then sits at zero until someone takes it.

When you take an **occupied** slot, 80% of what you pay becomes a claim for the outgoing tenure's miner, and 20% becomes protocol revenue. When you take an **empty** slot, there is nobody to compensate, so 100% becomes protocol revenue. There is no team fee anywhere in this. The outgoing tenure miner's 80% is held as a claim they withdraw when they like — anyone can trigger the withdrawal, but the money can only ever go to the miner.

Mine requests a nominal transfer of that protocol-revenue share into **ResonanceRouter** and stops. Under the supported standard USDG model, its `RevenueDeposited` event means the `SafeERC20` transfer request succeeded; it does not mean the money entered Resonance's stream in the same transaction.

Two things a prospective miner should understand. First, your GBX rate is **locked for your entire tenure** — halving issuance, redemptions, and other slots' replacements never change it. Only a newly occupied or replaced slot receives the current global TPS divided by sixteen. Second, less comfortably: **the 80% replacement claim is not guaranteed**. You receive it only if a later replacement clears at a nonzero price, and since the price falls to zero after an hour, any caller — including you — can replace the tenure having paid nothing. You keep the GBX you accrued, but receive no replacement claim. Interfaces must not present that 80% as principal, yield, or a refund.

16 PERMANENT SLOTS

fixed at construction · no owner · no capacity setter

held 1.00×	held 1.00×	for sale price falling	held 0.50×
held 1.00×	held 0.50×	held 0.50×	held 0.25×
held 0.50×	empty deposits 100%	held 0.25×	held 0.25×
held 0.25×	held 0.25×	held 0.25×	held 0.25×

Each slot runs its own auction, falling to zero over 1 hour, and pays 80% to whoever it displaces.

A slot's issuance rate is fixed when it is taken and never moves again. The mixed rates above are incumbents who took their slots under different halvings.

THE MINING MARKET IS A FIXED BOARD OF SIXTEEN INDEPENDENT SLOTS. NOTHING CAN ADD A SEVENTEENTH, AND NO SLOT'S RATE CAN BE CHANGED BY ANYONE ONCE IT IS TAKEN.

OCCUPIED-SLOT REPLACEMENT

80% is a claim the outgoing tenure miner withdraws.
20% is deposited in Router for later routing.

EMPTY SLOT — NOBODY TO COMPENSATE

MINING REPLACEMENT. STARTING A NEW OCCUPIED-SLOT TENURE COMPENSATES THE OUTGOING MINER; TAKING AN EMPTY SLOT FUNDS THE PROTOCOL ENTIRELY.

External LP token. A reviewed, externally created fungible Uniswap v2-style USDG/GBX LP token may be registered during bootstrap as an ordinary Strategy target. That is an index-asset choice, not a second revenue mechanism: the Strategy acquires and settles the LP token under the same global Fund/Bribe split as every other asset. The core does not create, seed, custody, price, rebalance, compound, harvest, or swap liquidity, and it guarantees no market liquidity.

7. How Resonance directs revenue over time

Revenue does not get handed out the moment it arrives. It flows into a contract called **Resonance**, which releases it as a **rolling seven-day stream**.

Think of it as a tank set to drain evenly over seven days. Whatever comes out at each moment is divided among Strategies in proportion to the sGBX signaling them *at that moment*.

Two design details exist to prevent specific attacks.

Revenue waits in a router until someone advances it. New revenue accumulates in a staging contract called **ResonanceRouter**. Anyone can call `route()`; if the balance cannot sustain one raw unit per second, or during an active period is smaller than the amount still scheduled, it stays put. When someone calls after it qualifies, the router forwards *everything*, and Resonance combines the new money with the ordinary Synthetix leftover and restarts a fresh seven-day stream. So restarting the stream early is possible but expensive — you must match what's left. It also means a mining payment can sit in the router for a while before it appears in the stream—or indefinitely if nobody calls. There is no keeper role or bounty; a frontend, volunteer keeper, or cron job is optional convenience infrastructure. A future mine-and-route helper could live in that periphery, but Mine itself must remain correct even if routing fails. Interfaces must show Router deposit and stream entry as different states.

Streaming is lazy. Entitlement accrues with time in the arithmetic, but tokens move only when someone triggers a settling action — a signal change, a revenue notification, a payout, or a purchase. So a Strategy's visible token balance understates what it is owed, and any interface that reads the raw balance and calls it inventory is wrong.

8. A numeric walkthrough: how signal allocation works

Let's make this concrete. All figures below are exact integer arithmetic at six decimals for USDG and eighteen for sGBX.

Setup. Resonance receives a notification of **604,800 USDG**. Seven days is 604,800 seconds, so the stream rate works out to exactly **1 USDG per second** with no rate remainder. When the division is not clean, the ordinary Synthetix whole-unit rate rounds down and the residue remains unallocated in Resonance.

Three participants and three Strategies:

PERSON	SIGNALS	SGBX MINTED
Ana	1,000 GBX to Strategy A (wrapped bitcoin)	1,000
Ben	3,000 GBX to Strategy B (staked ETH)	3,000
Cara	600 GBX to Strategy C (a governance token)	600

Total active signal weight: $1,000 + 3,000 + 600 = 4,600$ sGBX — which is also the entire sGBX supply, since none of it can sit idle.

Day 1 streams 86,400 USDG (86,400,000,000 raw units), split by weight:

STRATEGY	WEIGHT	SHARE	DAY 1 USDG
A	1,000	5/23	18,782.608695
B	3,000	15/23	56,347.826086
C	600	3/23	11,269.565217
			86,399.999998

Notice the total: **0.000002 USDG short**. That residue is the floor of the integer division. It is not paid to anyone, does not go to the treasury, and stays in Resonance permanently. This is a deliberate, documented trade-off — section 15 returns to it.

Day 2. Ben decides staked ETH is the wrong bet and reallocates his entire 3,000 signal from Strategy B to Strategy A through a removal and addition. Crucially, those calls settle Day 1 first — Ben's reallocation does **not** claw back the 56,347.826086 USDG that already accrued to Strategy B. From now on:

STRATEGY	WEIGHT	SHARE	DAY 2 USDG
A	4,000	20/23	75,130.434782
B	0	0	0
C	600	3/23	11,269.565217

Day 3. Cara withdraws her 600 signal from Strategy C — which in one atomic step removes the signal, burns her 600 sGBX, and returns her 600 GBX. Active weight is now just Ana's 1,000 plus Ben's 3,000, both on Strategy A:

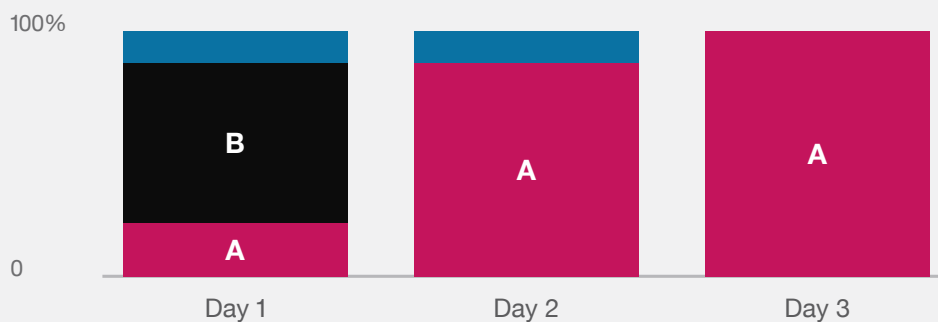
STRATEGY	WEIGHT	SHARE	DAY 3 USDG
A	4,000	100%	86,400.00

Totals after three days:

STRATEGY	ACCUMULATED USDG
A	180,313.043477
B	56,347.826086
C	22,539.130434

WHERE ONE DAY OF REVENUE GOES

Each day is the same 86,400 USDG, split by whoever is signaling at the time.



Day 2: Ben moves to A — Day 1 is already settled, so his move changes only later flow.

Day 3: Cara withdraws entirely, so A takes everything.

THE WORKED EXAMPLE FROM THE TEXT: THE SAME DAILY REVENUE, REDIRECTED AS SIGNALERS CHANGE THEIR MINDS.

Four days and 345,600 USDG remain in the stream. Cara earned Bribe rewards on Strategy C for two days and stopped the moment she withdrew. Ben's Strategy B holding is frozen at what it earned — it will accumulate nothing further unless someone signals it again.

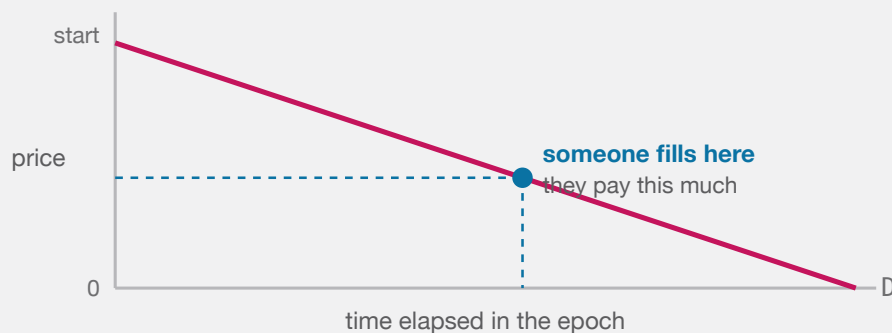
9. What a Strategy auction does

A Strategy now holds USDG and needs to convert it into the asset it is mandated to acquire. It does this with a **descending-price auction** — the design is sometimes called a “reverse Dutch auction” after the lineage it derives from; mechanically, the price starts high and falls until somebody takes the trade.

Here is the whole mechanism. The Strategy offers its **entire** USDG balance. It asks to be paid a certain quantity of the target asset. That asking quantity falls in a straight line to zero over a configured period (at minimum one hour, at maximum one year). Anyone may fill it at any moment by paying the current asking price. Once filled, the next auction starts at the price just paid multiplied by a fixed multiplier — so a hot auction restarts higher — subject to a floor and a ceiling.

There is no oracle. The auction is the price discovery. If the Strategy is asking too much, nobody fills it and the price falls. If it is asking too little, someone fills it immediately and the next auction starts higher.

Buyers are protected by three parameters on their own transaction: the auction round they expect (so a competing fill can't change the price under them), a deadline, and a maximum they will pay.



The next epoch restarts at the price just paid $\times$ a fixed multiplier, so a hot auction opens higher and an unfilled one keeps falling to zero.

BOTH THE MINING SLOTS AND THE STRATEGY AUCTIONS USE THE SAME SHAPE: THE ASKING PRICE FALLS IN A STRAIGHT LINE TO ZERO, AND WHOEVER THINKS IT IS CHEAP ENOUGH FILLS IT.

10. A numeric walkthrough: acquisition and redemption

Continuing the example, **Strategy A** holds **180,313.043477 USDG** and is mandated to acquire wrapped bitcoin.

The auction. Strategy A's auction runs over 24 hours with a starting price of **4 WBTC**. The price falls linearly. A buyer watching decides that 180,313 USDG is worth roughly 2 WBTC and waits.

At 13.2 hours elapsed (47,520 of 86,400 seconds, so 55% of the way through), the price has fallen 55%:

$$\text{price} = 4 \text{ WBTC} - (4 \text{ WBTC} \times 47,520 \div 86,400) = 4 - 2.2 = 1.8 \text{ WBTC}$$

The buyer fills. They pay **1.8 WBTC** and receive **180,313.043477 USDG**. (The purchase first pulls any revenue released to the Strategy through that exact timestamp, so they receive everything the Strategy is owed, not just its visible balance.)

Settlement — the default 90/10 setting. With Resonance's prospective `bribeBps` at its 10% deployment default, the 1.8 WBTC is classified as:

Fund	90% (default)	→	1.62 WBTC	(treasury backing for every GBX holder)
Bribe	10% (default)	→	0.18 WBTC	(reward for Ana and Ben, who signaled Strategy A)

This walkthrough uses the default, not an immutable split. The Resonance owner may set the Bribe share for later payments anywhere from 0% through 20%; Fund receives the 100%-minus-Bribe complement, or 80% through 100%. A change cannot reclassify this payment or any earlier purchase.

Strategy performs this split itself. The Fund share is transferred directly as part of the purchase, while the Bribe share goes to a small BribeRouter buffer for later permissionless distribution. A failed Fund transfer therefore reverts the complete auction purchase. A later Bribe notification failure does not: that share remains buffered.

DEFAULT ACQUISITION CLASSIFICATION

90%

10%

Fund

treasury backing for every GBX holder

Signalers

reward for signaling this Strategy

Each purchase floors its Bribe share; Strategy sends the complement directly to Fund.

THE DEFAULT IS 90% FUND / 10% BRIBE. STRATEGY FLOORS EACH PURCHASE'S BRIBE SHARE INDEPENDENTLY, SENDS FUND'S COMPLEMENT DIRECTLY, AND BUFFERS ONLY THE BRIBE SHARE.

Each purchase rounds its own Bribe share down. At the 10% default, ten separate one-row-unit payments therefore send all ten units to Fund, while one ten-unit payment sends nine units to Fund and one to Bribe. The protocol accepts this partition-dependent dust instead of maintaining another carry ledger.

The next auction round then starts at $1.8 \text{ WBTC} \times$ the configured multiplier — at $1.5\times$, that is **2.7 WBTC**.

So, at the default setting, Ana and Ben are paid directly out of the acquisition. They split 0.18 WBTC in proportion to their signal — Ana $1,000/4,000$ and Ben $3,000/4,000$, so 0.045 and 0.135 WBTC respectively. On top of that they may also earn separately funded **Bribes** — see section 12.

Redemption. Time passes. Many auctions run. Suppose the Fund now holds **50 WBTC** and **400 ETH**, GBX total supply is **100,000,000**, and miners have accrued **1,000,000 GBX** that has not yet been minted.

Diego holds 250,000 GBX and wants out. He calls redeem, naming WBTC and ETH.

Step 1 — pending mining counts first. Before doing anything else, the Fund reads Mine's effective supply: 100,000,000 minted GBX plus 1,000,000 accrued unminted GBX, or **101,000,000 GBX**. This constant-time view does not mint or touch every slot, and stops a redeemer from ignoring GBX the miners have already earned.

Step 2 — one snapshot for everything. The Fund records supply-before-burn = 101,000,000 GBX and records its balance of each named asset. Every payout uses this same denominator.

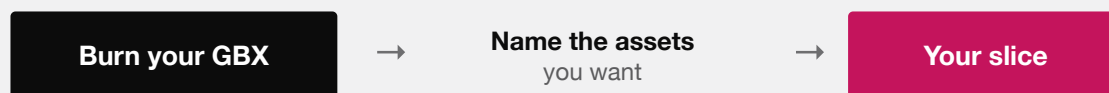
Step 3 — the payouts.

WBTC: $\text{floor}(50 \text{ WBTC} \times 250,000 \div 101,000,000) = 0.12376237 \text{ WBTC}$

ETH: $\text{floor}(400 \text{ ETH} \times 250,000 \div 101,000,000) = 0.990099009900990099 \text{ ETH}$

Step 4 — the burn. Diego's 250,000 GBX is burned. Supply falls to 100,750,000 GBX. Everything above happens in one atomic transaction: if any transfer fails, the entire redemption including the burn is undone.

REDEEMING



For each asset you name:

$\text{floor}(\text{Fund's balance} \times \text{your GBX} \div \text{total GBX supply})$

The supply is snapshotted after every miner is credited, so nobody is diluted by unminted rewards. Assets you leave out are forfeited — permanently, to everyone else.

It is one atomic step. If any transfer fails, the burn is undone too.

REDEMPTION IN FULL. NO QUEUE, NO GATE, NO DISCRETION — ONLY ARITHMETIC AGAINST A SNAPSHOT.

Two things worth noticing. If Step 1 ignored pending mining, Diego's WBTC payout would have been 0.125 WBTC instead of 0.12376237 — the difference is exactly the dilution from recognising the miners' claim, and it is correct. And if the Fund had been holding, say, 2,000,000 GBX from a GBX-denominated auction that nobody had burned yet, that GBX would have counted in the denominator and reduced Diego's payout. Anyone can burn Fund-held GBX at any time, and a redeemer should do so first.

11. What the Fund holds

The Fund is an **ownerless raw-token treasury**. Not a curated portfolio — a treasury.

It has no administrator, no roles, and no asset registry. Assets can leave in exactly two ways: a GBX holder redeems, or someone burns Fund-held GBX. There is no sweep, rescue, recovery, or migration function of any kind.

The consequence of having no registry is that **anyone can send any ERC-20 to the Fund and it becomes redeemable backing**, reviewed or not. Being in the Fund does not make a token official. Official membership means having a registered Strategy — nothing more. That registered set is the closest thing the protocol has to an index: it is the curated list of assets the protocol is trying to acquire. But it is only a list. There are no target weights, no rebalancing, and no valuation — whatever proportions the Fund ends up holding are an outcome, not a plan. Interfaces must label unsolicited Fund balances separately.

Because you name the assets you want, you can skip a broken or worthless token rather than have it block your redemption. The trade-off is that **anything you skip is permanently forfeited** — it stays in the Fund for the remaining supply. There is no partial-claim ledger.

12. How signalers are rewarded

Signalers have two stacked income sources, and they arrive through the same contract.

First, the automatic acquisition share — 10% by default. Every time their Strategy acquires an asset, the prospective global Bribe share then active on Resonance is streamed to whoever is signaling it, split in proportion to signal. The owner can set that share from 0% through 20%; the Fund receives the complement. It also aligns the incentive neatly: when the share is nonzero, you earn the asset you voted to accumulate, so signaling for something worthless pays you in something worthless.

Second, Bribes. A “bribe,” here, is not corruption — it is the standard term for an open, permissionless reward stream used to attract attention. Anyone can deposit additional reward tokens into a Strategy’s pool, which streams them to that Strategy’s signalers the same way.

Who would do that? A project that wants the protocol to accumulate its token; a DAO that wants its asset in a long-term treasury; a market maker who benefits from the auction flow. The protocol takes no position on motive.

Some deliberate details, which apply to both sources:

- **At most sixteen reward tokens per Strategy**, hard-coded, append-only. The cap exists so that entering, leaving, and settling a signal position stay bounded in gas no matter what anyone registers.
- **Reward accounting follows the Synthetix shape.** A qualifying top-up combines with the scheduled amount left and restarts seven days. It must be large enough to avoid a zero rate and cheap permissionless stream slowing.
- **If everyone stops signaling, reward time continues.** That interval remains unallocated token surplus rather than being queued for later signalers.
- **You can claim one token at a time**, so a single frozen reward token cannot block the rest.
- **Rewards you arrived too late for are not yours.** Rate, index, and account division floors remain unallocated in the Bribe rather than being reassigned through carry buckets.
- **Each reward token has a lifetime ceiling on how much can ever be streamed through it.** The number is astronomically large — for a normal eighteen-decimal token, far more than any real supply — and exists purely so that a token with an absurd unit count cannot be used to jam the reward arithmetic and trap other people’s deposits. Reaching it would block further funding of that one token only; existing rewards, claims, and withdrawals keep working.

13. How governance works — and what is still missing

Start with the good news, because it is the larger part. There are exactly **four things** about this protocol that anyone can ever change:

1. Add a Strategy.
2. Permanently retire a Strategy.
3. Register a Bribe reward token (within the sixteen-token cap).
4. Set the signalers’ share of each acquisition, anywhere from 0% to a hard ceiling of 20%.

NO CONTINUING CUSTOM OWNER AUTHORITY

No local custom owner-administration calls.

Mine	sixteen slots, fixed
Fund	the treasury itself
GBX	minter locked once
Strategy	auction parameters fixed
BribeRouter	Bribe-only qualifying buffer

CONTINUING CUSTOM OWNER AUTHORITY

Resonance — four custom protocol calls:

- add a Strategy
- retire a Strategy
- register a reward token
- set Bribe share (0–20%)

The holder of that address is
not yet chosen.

There is no upgrade path, pause switch, sweep, or migration route anywhere in the protocol — so this map is the complete authority surface, not a summary of it.

SignalGBX, StrategyFactory, and BribeFactory retain setup-only Ownable shells until production explicitly renounces them; after binding, those owners have no custom protocol call.

THE COMPLETE CUSTOM PROTOCOL AUTHORITY SURFACE: FOUR BOUNDED CALLS ON RESONANCE, WITH NO LOCAL ADMINISTRATION ON THE CONTRACTS AT LEFT. INHERITED OWNERSHIP TRANSFER AND RENUNCIATION REMAIN; WHO HOLDS THAT OWNER ADDRESS IS THE LARGEST OPEN QUESTION.

That is the complete list. All four live on **Resonance**, which is the only contract with continuing custom owner authority. SignalGBX, StrategyFactory, and BribeFactory retain setup-only Ownable shells until production explicitly renounces them after their one-time Resonance bindings are consumed; those shells expose no custom protocol action after setup. The Mine has no owner. The Fund has no owner. The core has no liquidity position. Nobody — not a developer, not a voter, not a future administrator — can change mining prices, issuance rates, halving parameters, the tail rate, mint authority, Fund assets, liquidity custody, the auction mechanism, or the sixteen-slot count. There is no upgrade path, no pause switch, and no sweep function to add one later.

The fourth item is the one genuine economic dial, and it is deliberately fenced. The reward share starts at 10% and can never exceed **20%**, so at least 80% of every acquisition reaches the treasury no matter who holds the owner address. A change applies only to purchases settled after it — it cannot reach back and reclassify an amount already recorded, a reward already streaming, or anything already in the Fund.

There is one guard rail on retirement: **the final live Strategy cannot be retired**. If only one remains, the call reverts. To replace it, the replacement must be added first. This guarantees there is always somewhere valid to signal, which matters because signaling is the only way to hold sGBX at all.

The part that is not finished

Those four actions sit behind a **single owner address** on Resonance. Who holds that address has not been decided.

The protocol used to ship its own voting contract and its own timelock. Both were removed ([ADR 0034](#)), because the intended deployment will hand ownership to an established external governance system instead — and maintaining a second, home-grown voting stack that was never going to be deployed meant carrying security surface for nothing. Removing it was the right call. But it does mean that today the protocol contains **no voting rules, no quorum, no proposal filter, and no execution delay of its own**. What sGBX offers is the vote checkpoints described in §5, sitting ready for a system that has not yet been attached.

Concretely, in this development tree, whoever holds the Resonance owner address can add a Strategy, retire a Strategy, register a reward token, or call `setBribeBps` to set the prospective automatic reward share from 0% through 20% immediately — no vote, no waiting period, no way for anyone to object. The rate change cannot re-classify earlier purchases. The owner can also hand that address to someone else, or throw it away permanently. In development that address is simply the deployment fixture.

Four honest limitations follow:

- **Governance is a placeholder, not a mechanism.** Until an external system is chosen, reviewed, and actually given ownership, there is no meaningful answer to “who decides.” Deploying without finishing that step would produce a protocol with an ordinary admin key — precisely the thing this design exists to avoid.
- **The protocol guarantees no delay, no veto, and no cancellation.** Whatever protections eventually exist will be properties of the external system, not of this code, and they will need reviewing on their own terms.
- **Retiring a Strategy is irreversible**, which makes both points above materially more serious.
- **Vote checkpoints survive withdrawal** (§5), so any system attached later must decide deliberately how much weight to give historical balances.

14. What happens when something fails

The protocol is built so a failure in one place doesn’t cascade:

- **A broken token cannot trap your position.** Withdrawing signal is pure accounting — it never transfers a reward, payment, or revenue token, so a frozen third-party token cannot block them.
- **A frozen Fund can block its own auction purchase.** The Strategy pays Fund directly, so the purchase is atomic with successful Fund receipt.
- **A broken asset cannot block redemption for everyone else.** You name what you want, so you can omit it.
- **Only standard, non-rebasing ERC-20s are supported.** SafeERC20 checks call success, but Mine, SignalGBX, reward, and settlement paths deliberately do not duplicate pre/post balances. Canonical GBX/USDG and governance-registered tokens must not use fee-on-transfer, rebasing, or hostile behavior. Fund retains stricter guards for caller-selected arbitrary redemption assets.
- **A failed automatic Bribe notification stays buffered.** It cannot undo the completed purchase or redirect the tokens elsewhere.

One failure mode has no recovery. When a Strategy is retired, its reward pool stays open for existing signalers but nobody new can join. If the **last** signaler withdraws while rewards remain, those rewards are stranded permanently — possibly an entire unvested stream, not merely dust. There is deliberately no refund or rescue. Interfaces must warn the final signaler before they exit.

15. What GumBall6900 does not guarantee

Read this section twice.

- **It does not guarantee that assets appreciate.** The protocol buys what signalers point it at. That aggregates conviction; it does not make conviction correct.
- **It does not guarantee investment returns.** There is no yield, distribution, or promise of any kind. Your redemption is a share of whatever the Fund happens to hold, which may be worth less than you paid.

- **It does not guarantee auction liquidity.** An auction fills only if someone chooses to fill it. A price falling to zero guarantees a *price*, not a *buyer*.
- **It does not guarantee good signal choices.** There is no quality filter beyond whoever approves a Strategy, no diversification requirement, and no risk framework.
- **It does not guarantee safe external tokens.** USDG and every payment and reward token are third-party contracts that may be upgradeable, pausable, blocklisting, or malicious. The Fund accepts any ERC-20 without review.
- **It does not guarantee permanent frontend availability.** The contracts are permissionless, but no website, indexer, or API is guaranteed to exist. You may need to interact with contracts directly.
- **It does not eliminate smart-contract risk.** The code has not been independently audited, and because nothing is upgradeable, a discovered bug cannot be patched.
- **It does not eliminate governance risk.** The system that will own Resonance is unselected, so capture, delay, veto, and accountability are all open questions rather than settled properties (section 13).
- **It does not eliminate chain, MEV, or timing risk.** Auctions and mining slots are competitive, publicly visible opportunities. Transactions can be reordered, front-run, or censored, and chains can reorganize.
- **It does not eliminate regulatory risk.** Legal treatment in any jurisdiction is unresolved.
- **And it accumulates permanent dust.** Rounding residue and revenue streamed while nobody signals accrue in Resonance forever with no recovery path. Precision keeps individual amounts tiny, but **no lifetime bound on the total is claimed.**

16. Major risks, summarized

RISK	WHAT IT MEANS
Incomplete external review	V12 supplied a commit-pinned finding export; two reproduced behaviors are accepted constraints and claim authorization remains open.
Immutability	No patch, no pause, no rescue. A bug or deployment error is permanent.
Deployment correctness	Parameters, reviewed Strategy inputs, and role setup must be right the first time, forever.
Unresolved economics	The provisional 64 GBX/s, 69-day, 1 GBX/s Mine schedule and other fixed economics lack independent review.
Unfinished governance	The external owner of Resonance is unselected; today one address holds all four powers outright.
Miner rollover	The 80% replacement claim exists only if a later replacement pays. It can be zero.
Abandoned rewards	A retired Strategy's last signaler can strand an unbounded amount of rewards.
Accepted dust	Rounding and zero-signal intervals accumulate unrecoverable USDG in Resonance.
Third-party tokens	USDG and every acquired asset carry independent freeze, upgrade, and solvency risk.
Legal and provenance	Upstream code lineage and license reconciliation are unresolved release blockers.

17. Current project status

To be exact about the current development candidate and V12 review target:

- **Not deployed.** No contract is live on any network. No signed deployment manifest exists. The intended target chain and the canonical USDG address remain unresolved candidates. Any bootstrap LP token address is a reviewed deployment input, not a hard-coded protocol address.
- **External review remains incomplete.** V12 supplied 22 Low candidate findings for reviewed source commit 3ae171b . Our independent disposition confirmed 249695, 249702, and 249705, while the export lacks a full scope, methodology, named auditor, date, signature, and final assurance statement. Symbolic analysis and formal verification also remain incomplete.
- **ADR 0051 is outside that review.** Its renamed scalar selectors, batch loops, aggregate custody transitions, stateless portfolio Lens, direct-call SDK composition, and subgraph position index are later development work and require fresh independent review.
- **The full deterministic workspace matrix passed locally before ADR 0045.** The uncommitted ADR 0044 tree passed 356/356 default Foundry tests across 25 suites, 19/19 integration tests across two suites, Hardhat 4/4, SDK 50/50, TypeScript simulations 39/39, Python environment-policy checks 5/5 and simulations 25/25, subgraph specification checks 4/4 plus Matchstick 10/10 and build, web unit tests 3/3, Playwright 6/6, and the documentation, ABI, formatting, lint, typecheck, and workspace-build gates. This is unpinned local engineering evidence, not an audit or release approval.
- **Static analysis, mutation testing, and external fuzzing are pinned to older trees.** Those campaigns remain useful engineering history, but they predate ADR 0044 and are **not** current evidence for the code described in this article. Re-running and manually reviewing them remains open.
- **Four High-severity release gates remain open.** Two concern deployment: Mine's selected, hard-coded, and modelled economics have not received independent review, and no signed manifest yet proves the deployed bytecode, constructor arguments, and dependency addresses. Two concern governance: the external system that will own Resonance is unselected, and its voting, delegation, and delay semantics therefore remain unreviewed. Also outstanding: a second external-fuzzer seed, a monitored testnet rehearsal, and release review.
- **Legal and provenance clearance is outstanding.** The protocol's contracts adapt several upstream codebases whose licensing chain has not been resolved, including one with a GPL ancestor.

Nothing in this article should be read as a claim that the protocol is safe, audited, live, launched, production-ready, or suitable for funds you cannot afford to lose.

See also: the [one-pager](#) for a two-minute summary, and the [technical whitepaper](#) for exact formulas, state machines, accounting identities, and the full threat model.